
Generalized Additive Models

ECON 8310: Business Forecasting — Lecture 3

Department of Economics

University of Nebraska at Omaha

August 24, 2026

Lecture Outline

- 1 Motivation: Beyond Linear, Without Black Boxes
- 2 GAM Structure and Smoothing
- 3 Prophet for Time Series GAMs
- 4 pyGAM for General Additive Modeling
- 5 Key Takeaways and Roadmap

Motivation: Beyond Linear, Without Black Boxes

We want flexibility to capture nonlinear relationships while still being able to *explain* what the model is doing.

The Flexibility–Interpretability Tradeoff

Linear models (ARIMA, regression):

- ✓ Each coefficient has a clear meaning
- ✓ Stable, well-understood properties
- ✗ Assume a constant slope everywhere
- ✗ Miss curvature and threshold effects

Fully nonlinear models (trees, neural nets):

- ✓ Capture almost any pattern
- ✓ High accuracy on complex data
- ✗ Hard to explain any single prediction
- ✗ Interactions entangle all variables

GAMs sit between the two. They allow a nonlinear, flexible relationship for each variable, but keep the model **additive** — so each variable's effect can still be plotted and explained on its own. That is the whole bargain, and the rest of this lecture is about what it costs.

Why Additivity Matters for Interpretation

In a non-additive model the effect of one variable depends on the value of another, so you cannot describe them separately.

Concretely. Suppose the truth is $y = x_1^2 x_2^2 + \varepsilon$. Then $\partial y / \partial x_1 = 2x_1 x_2^2$ — it depends on *both* variables. There is no answer to “what does x_1 do to y ?” that does not begin with “it depends on x_2 .”

GAM Marginal Effects Are Simple

In a GAM, $y = f_1(x_1) + f_2(x_2) + \varepsilon$, so the marginal effect of x_1 is just $f'_1(x_1)$ — a function of x_1 alone, whatever x_2 happens to be.

That is why you can plot each variable's effect separately and hand the plot to a stakeholder.

GAM Structure and Smoothing

Additive nonlinear functions, penalized for wiggleness.

What Is a GAM?

Generalized Additive Model

$$y = \alpha + f_1(x_1) + f_2(x_2) + \cdots + f_k(x_k) + \varepsilon$$

where each $f_j(\cdot)$ is a **smooth, nonlinear function** estimated from the data (Wood 2017).

Each f_j is free to be linear, curved, U-shaped, or threshold-like — the data decides. Because the terms are *added* rather than multiplied, the effects stay independent and each one can be visualized on its own. Every function is fitted to minimize prediction error subject to a smoothness penalty, which is the next slide.

A retail example. Model weekly sales as $f_1(\text{time})$ for the long-run trend, plus $f_2(\text{week of year})$ for seasonality, plus $f_3(\text{unemployment})$ for economic sensitivity. Three curves, each plotted and interpreted separately — and none of them requiring you to hold the others fixed to make sense of it.

What the “Generalized” Actually Means

So far every y we have modelled has been continuous and roughly symmetric — sales in dollars, demand in units. For that case an *additive* model is enough. The **G** in GAM is what happens when y is not that kind of variable.

Generalized Additive Model, in full

$$g(\mathbb{E}[y]) = \alpha + f_1(x_1) + \cdots + f_k(x_k)$$

The additive predictor models a *transformation* of the mean. That transformation $g(\cdot)$ is the **link function**.

If y is...	Distribution	Link $g(\cdot)$
Continuous, symmetric (sales)	Gaussian	Identity — $g(\mu) = \mu$
A count (units, visits, claims)	Poisson	Log — keeps the forecast positive
Binary (churn, stockout, yes/no)	Binomial	Logit — keeps it inside $[0, 1]$

In pyGAM this is the difference between LinearGAM, PoissonGAM, and LogisticGAM — same additive structure, different link. Everything else in this lecture is unchanged.

Controlling Smoothness: The Penalty Term

A GAM minimizes a **penalized** sum of squared errors:

$$\text{PSSE} = \underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{fit}} + \underbrace{\lambda \int (f''(x))^2 dx}_{\text{smoothness penalty}}$$

The second derivative $f''(x)$ measures how fast the slope is changing, so a wiggly function accumulates a large integral and a smooth one accumulates a small one. The penalty therefore charges the model for wiggleness, and λ sets the price.

The two extremes are worth holding in mind. As $\lambda \rightarrow 0$ the penalty vanishes and the curve is free to chase every point — overfitting. As $\lambda \rightarrow \infty$ every f_j is forced straight and the GAM collapses back into ordinary least squares. Everything useful happens in between, and λ is chosen by **cross-validation** or GCV rather than by eye.

Splines are how $f(x)$ is represented: piecewise polynomials joined smoothly at **knot** points.

Prophet for Time Series GAMs

Facebook's Prophet decomposes a time series into trend, seasonality, and holiday effects using a GAM.

Prophet: Forecasting at Scale

Taylor and Letham (2018) built Prophet for large-scale forecasting at Meta. It is a GAM in time-series clothing — the series is decomposed additively:

$$y(t) = \underbrace{g(t)}_{\text{trend}} + \underbrace{s(t)}_{\text{seasonality}} + \underbrace{h(t)}_{\text{holidays}} + \varepsilon_t$$

Trend $g(t)$ is piecewise linear or logistic, with *changepoints* where the slope is allowed to shift. **Seasonality** $s(t)$ is a Fourier series, which is how Prophet handles weekly and yearly cycles simultaneously. **Holidays** $h(t)$ are irregular events you supply yourself — Black Friday, a product launch, a lockdown.

The design goals were practical rather than statistical: one fit-and-forecast call, robust to missing data and outliers, and interpretable because every component can be plotted on its own. That is why it spread so quickly among analysts who are not statisticians.

changepoint_prior_scale controls trend flexibility: high adapts quickly and risks overfitting; low keeps the trend stable.

Prophet on Retail Sales

```
from prophet import Prophet
df = pd.DataFrame({'ds': dates, 'y': sales})
m = Prophet(changepoint_prior_scale=0.05, yearly_seasonality=True).fit(df)
future = m.make_future_dataframe(periods=12, freq='M')
forecast = m.predict(future); m.plot_components(forecast)
```

`plot_components` is the call that matters here: it draws the trend with its changepoints, the yearly wave, and the day-of-week pattern as separate panels — the additive structure made visible.

Reach for Prophet on seasonal series with known holiday effects, especially when someone will ask you to explain the forecast. **Reach for pyGAM** when you have many correlated external predictors instead.

pyGAM for General Additive Modeling

When you have multiple predictors and want flexible nonlinear effects for each.

pyGAM: Flexible Additive Modeling

pyGAM fits a GAM with a smooth function you specify per predictor:

pyGAM Example

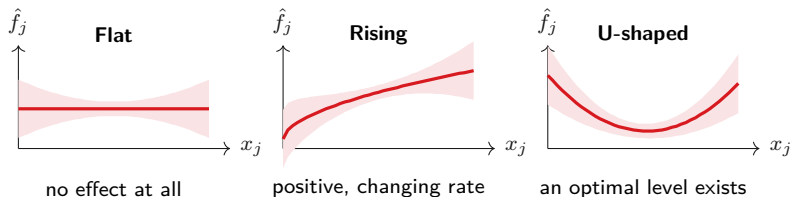
```
from pygam import LinearGAM, s, f, l
gam = LinearGAM(s(0) + s(1) + s(2) + f(3))
gam = gam.gridsearch(X, y)
pdep, confi = gam.partial_dependence(term=0, width=0.95)
y_hat = gam.predict(X_new)
```

The three term types are the whole vocabulary: $s(j)$ is a smooth spline of feature j — the flexible nonlinear effect; $l(j)$ forces a straight line, which is worth using when theory says the effect should be linear; and $f(j)$ handles a binary or categorical feature.

`gridsearch` then tunes λ by cross-validation — though you should still look at the resulting curves and ask whether they are plausible.

Interpreting GAM Results: Partial Dependence Plots

A partial dependence plot shows $\hat{f}_j(x_j)$ across the observed range, with confidence bands. Reading the shape is most of the interpretation.



Watch the **bands** as much as the curve — they widen where the data thin out, which is exactly where a reader is tempted to extrapolate.

The output is a sentence someone can act on: *"as unemployment rises from 5% to 10%, predicted sales fall by X % — and the decline steepens past 8%."* No linear model could tell them that second clause.

Key Takeaways and Roadmap

GAMs: flexible, interpretable, and underutilized in business forecasting.

GAMs vs. Other Methods: When to Choose Each

Method	Nonlinear	Interactions	Interpretable	Use case
ARIMA / ETS	No	No	High	Univariate time series
Linear regression	No	Manual	High	Few predictors
GAM	Yes	No	High	Few predictors, explain effects
Random Forest	Yes	Yes	Medium	Many predictors, tabular
XGBoost	Yes	Yes	Low	Best accuracy, tabular

The column that decides it is usually **Interactions**. A GAM is additive by construction, so if the effect of price genuinely depends on whether it is a holiday week, a GAM cannot represent that and a tree can.

Choose a GAM when you must explain each predictor's effect, when those effects are curved rather than straight, when a time series has complex trend and seasonality (reach for Prophet), and when interpretability matters more than the last few points of accuracy. That last clause is the honest one — on tabular data, XGBoost will usually win on RMSE, and Lecture 6 is where we take that seriously.

Lecture 3: Key Takeaways

1. **GAMs** offer nonlinear flexibility while maintaining interpretability:
 $y = \alpha + f_1(x_1) + f_2(x_2) + \dots + f_k(x_k) + \varepsilon$. Because the model is additive, each variable's effect can be visualized and explained independently.
2. **Smoothness penalty** (PSSE) prevents overfitting. The penalty λ controls the trade-off between fit and smoothness. It is chosen by cross-validation.
3. **Prophet** decomposes a time series into trend, seasonality, and holiday effects using a GAM. Ideal for business time series with seasonal patterns and known irregular events.
4. **pyGAM** fits a general GAM with any combination of predictors. Use `s()` for smooth effects, `l()` for linear, `f()` for categorical. `gridsearch` tunes λ automatically.
5. Partial dependence plots make results **actionable for stakeholders**: show clearly how each variable affects the forecast.

Next: Decision Trees — moving to tree-based methods that capture interactions automatically.

References I

-  Taylor, Sean J. and Benjamin Letham (2018). “Forecasting at Scale”. In: *The American Statistician* 72.1, pp. 37–45.
-  Wood, Simon N. (2017). *Generalized Additive Models: An Introduction with R*. 2nd. Boca Raton, FL: Chapman and Hall/CRC.